

JAXMg: A multi-GPU linear solver in JAX

Jacob Tutt^{1,2*} and Roeland Wiersema^{3*}

¹ Cavendish Astrophysics, University of Cambridge, Cambridge CB3 0HE, UK ² Kavli Institute for Cosmology, University of Cambridge, Cambridge CB3 0HA, UK ³ Center for Computational Quantum Physics, Flatiron Institute, 162 Fifth Avenue, New York, NY 10010, USA
* These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [↗](#)

Submitted: 03 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Solving large dense linear systems and eigenvalue problems is a core requirement in many areas of scientific computing, but scaling these operations beyond a single GPU remains challenging within modern programming frameworks. While highly optimized multi-GPU solver libraries exist, they are typically difficult to integrate into composable, just-in-time (JIT) compiled Python workflows.

JAXMg provides distributed dense linear algebra for JAX, enabling linear solves and decompositions for matrices that exceed single-GPU memory limits. By interfacing JAX with NVIDIA's cuSOLVERM through an XLA Foreign Function Interface, JAXMg exposes distributed GPU routines as JIT-compatible JAX primitives. This design allows scalable linear algebra to be embedded directly within JAX programs, preserving composability with JAX transformations and enabling multi-GPU and multi-node execution in end-to-end scientific workflows.

Statement of need

Modern scientific computing increasingly relies on GPUs, which now provide a large fraction of the available floating-point throughput in both supercomputers and smaller multi-GPU workstations. At the same time, dense linear algebra remains a critical building block for many numerical methods. A number of mature libraries therefore provide high-performance linear algebra on CPUs and GPUs, including distributed and accelerator-aware packages such as ScaLAPACK (Blackford et al., 1997), MAGMA (Abdelfattah et al., 2024), and SLATE (Gates et al., 2019).

In parallel, JAX (Bradbury et al., 2018) has become a widely adopted framework for scientific computing because it combines a simple user experience with JIT compilation and automatic differentiation. The JAX ecosystem has expanded rapidly, with libraries for neural networks (Heek et al., 2024), Bayesian inference (Cabezas et al., 2024), differential equations (Kidger, 2021), variational Monte Carlo (Vicentini et al., 2022), and full physics simulation environments (Freeman et al., 2021). These workflows often require repeatedly solving linear systems or computing eigenvalue decompositions, either as part of a larger simulation loop or inside differentiable optimization.

Despite this growth, and the availability of packages such as Lineax for composable linear solves within JAX (Rader et al., 2023), the ecosystem still lacks distributed dense linear solver routines that scale across multiple GPUs while remaining usable from idiomatic JAX programs. This gap makes it challenging to take existing JAX-based scientific applications to problem sizes that exceed a single GPU, or to integrate multi-GPU linear algebra into more complex JAX pipelines. Existing approaches require leaving the JAX execution model, either by exporting

arrays to external MPI-based solvers or by manually orchestrating GPU kernels outside JAX's JIT. These approaches break composability and complicate memory management.

JAXMg addresses this need by providing a distributed multi-GPU and multi-node interface to GPU-accelerated solver backends, enabling scalable linear solves and eigendecompositions from within JAX.

Software design

JAXMg connects JAX to NVIDIA's distributed dense linear algebra library cuSOLVERMg (Corporation, n.d.) via an XLA Foreign Function Interface (FFI) C++/CUDA extension. This design enables writing complex, JIT-compatible JAX programs while delegating the computationally intensive components to a compiled backend.

Simply pass JAXMg an ordinary JAX array sharded over a two-dimensional device mesh. The native backend handles the local memory-layout conversion, 2D block-cyclic redistribution, distributed solver execution, and restoration of the result to its original JAX layout.

The current release provides a JIT-compatible interface to four workflows:

- potrs: Solves $Ax = b$ for symmetric (Hermitian) positive-definite A using a Cholesky factorization (cusolverMgPotrf and cusolverMgPotrs). The same factorization can optionally return $\log \det(A)$.
- lu_solve: Solves $Ax = b$ for general nonsingular A using a pivoted LU factorization (cusolverMgGetrf and cusolverMgGetrs).
- syevd: Computes the eigenvalues λ_i and eigenvectors v_i of a symmetric (Hermitian) matrix A , satisfying $Av_i = \lambda_i v_i$ (cusolverMgSyevd).
- gesvd: Computes the singular-value decomposition of an $M \times N$ matrix $A = U\Sigma V^\dagger$, returning the singular values and optional left and right singular vectors (cusolverMgGesvd).

All four routines support the JAX dtypes float32, float64, complex64, and complex128, with CUDA 12 and CUDA 13 backends available for both x86_64 and aarch64 systems.

For example, the potrs routine can be called over a two-dimensional mesh:

```
mesh = jax.make_mesh((jax.process_count(), 1), ("pr", "pc"))
matrix_specs = P("pr", "pc")
```

Here, A is an $N \times N$ positive-definite matrix sharded over the process rows and columns. The solve input b has shape $N \times N_{\text{RHS}}$ and is sharded over the process rows:

```
A = jax.device_put(A, NamedSharding(mesh, P("pr", "pc")))
b = jax.device_put(b, NamedSharding(mesh, P("pr", None)))
```

The sharded arrays can then be passed directly to potrs:

```
out = potrs(A, b, T_A=T_A, mesh=mesh, matrix_specs=P("pr", "pc"))
```

The tile size T_A is user-configurable and controls the trade-off between memory usage and performance; larger tiles typically improve throughput once the problem is sufficiently large (see Figure 4).

Memory-efficient data redistribution

Parallelized linear algebra algorithms require a distributed data layout to ensure proper load balancing of the available computational power (Dongarra et al., 1994). For JAXMg, the central challenge is constructing this layout without reducing the matrix sizes that can be held in aggregate GPU memory. JAXMg therefore transforms the donated matrix buffers in place and reuses a single bounded scratch allocation across all stages. Minimizing memory overhead

81 alone, however, is not sufficient: the redistribution must also use the available interconnect
 82 bandwidth efficiently. Although arbitrary permutations can be decomposed into fine-grained
 83 cycles, repeated small transfers introduce synchronization and transfer overheads, leading
 84 to poor utilization of the bandwidth available from modern GPU interconnects (Li et al.,
 85 2020). JAXMg addresses both requirements through a three-stage redistribution. Each stage
 86 moves the largest contiguous regions that fit within a shared scratch allocation and performs
 87 independent transfers concurrently wherever dependencies allow. The size of this allocation is
 88 determined by the tile slabs used in the final 2D block-cyclic stage, described in Section 3.1.3,
 89 and is reused throughout. The following sections describe the three forward redistribution
 90 stages used to prepare the matrix for distributed solver execution; after the solver completes,
 91 these stages are reversed to restore the original JAX layout.

92 Local memory-layout conversion

93 The first stage reconciles the physical memory layouts used by JAX and cuSOLVERMp. JAX
 94 stores each local matrix shard in row-major order, whereas cuSOLVERMp requires column-major
 95 local buffers. While XLA can materialize a column-major FFI input, doing so requires a second
 96 full-sized local matrix allocation, defeating the low-memory design. JAXMg instead applies a
 97 parallel implementation of the rectangular permutation decomposition introduced by Catanzaro
 98 et al. (2014) directly to each donated buffer. The method expresses the layout change as
 99 modular column and row permutations, processed in batches bounded by the shared scratch
 100 allocation. The logical matrix remains unchanged and, because the conversion is entirely local,
 101 this stage runs concurrently on every GPU without inter-device communication.

102 Edge-padding alignment

103 Due to the 2D block-cyclic layout required by the cuSOLVERMp backend, JAXMg pads a
 104 local JAX shard before it enters the native backend if either dimension is not divisible by
 105 the corresponding tile dimension. This provides enough local capacity for the solver layout,
 106 but leaves padding between neighbouring shards in the global process grid. As a result, the
 107 destination of a solver tile may still contain part of another tile, so the redistribution in Section
 108 3.1.3 cannot yet move complete tile slabs directly. JAXMg therefore compacts the real matrix
 109 towards the global top-left, leaving the padding on the global right and bottom edges, as
 110 illustrated in Figure 1.

111 The compaction proceeds in two passes. Column slabs are first shifted left within each
 112 process row, after which row slabs are shifted upwards within each process column. Since the
 113 padding provides empty destinations, these movements form open chains and do not require an
 114 additional temporary buffer for preserving overwritten data. Dependencies between movements
 115 prevent an entire pass from being executed at once, so each pass is divided into ordered
 116 waves. Within each wave, the largest slabs that fit in the shared scratch allocation are moved
 117 concurrently across independent process rows or columns.

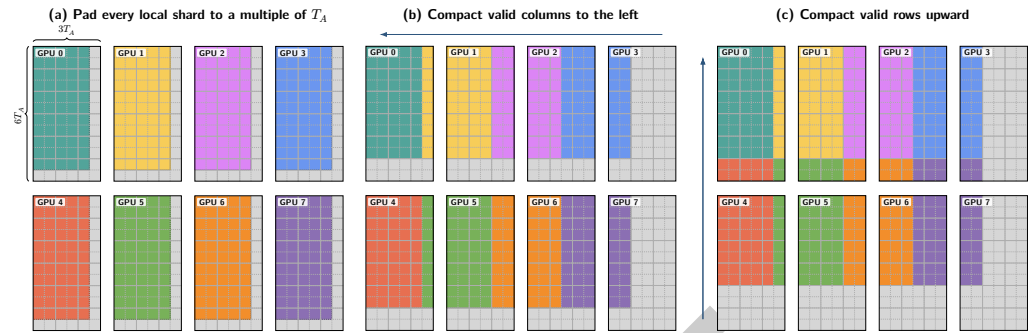


Figure 1: Demonstration of the edge-padding alignment across a 2×4 GPU process grid. (a) Initial JAX block-sharded layout after each local shard is padded to a multiple of T_A . (b) Horizontal compaction shifts valid column slabs to the left within each process row. (c) Vertical compaction then shifts valid row slabs upwards within each process column, leaving all padding on the global right and bottom edges. Colours indicate the GPU that originally owned each matrix entry, while grey cells denote padding.

118 Unlike the in-place redistribution handled by the native backend, this initial capacity padding
 119 must be performed by JAX because an existing donated allocation cannot be expanded once
 120 assigned. Materializing the padded array thus temporarily requires both the original and padded
 121 buffers, reducing the matrix size that fits in available GPU memory. Padding should therefore
 122 be avoided where possible by choosing a tile size that divides both dimensions of every local
 123 matrix shard.

124 2D block-cyclic redistribution

125 Finally, JAXMg constructs the 2D block-cyclic layout required by cuSOLVERMp. For a process
 126 grid with P_r rows and P_c columns, tiles are distributed in round-robin order over both axes,
 127 such that the tile at global tile coordinate (i, j) is assigned to

$$\text{owner}(i, j) = (i \bmod P_r, j \bmod P_c).$$

128 JAXMg constructs an explicit mapping from every source tile to its destination and applies
 129 it in two separable phases. First, the column-owner mapping is decomposed into disjoint
 130 permutation cycles that rotate complete tile-column slabs within each process row. The
 131 corresponding cycles run concurrently across all process rows until every tile has the correct
 132 process-column owner. The same procedure is then applied at the row level within each process
 133 column and parallelized across the process columns, as shown in Figure 2.

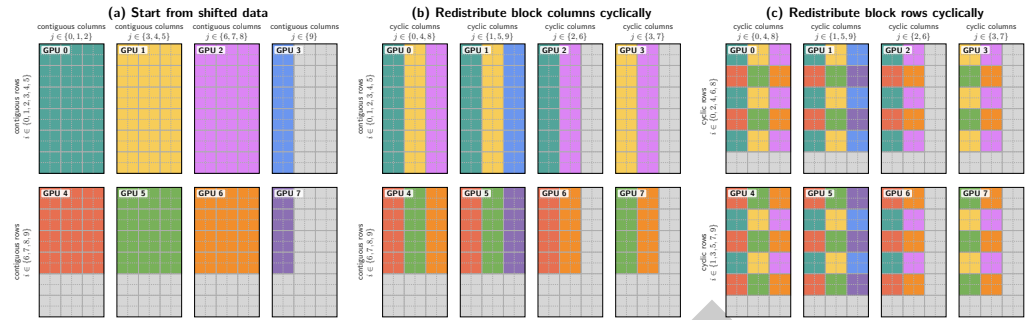


Figure 2: Demonstration of the redistribution from the compacted JAX block-sharded layout to the 2D block-cyclic layout required by cuSOLVERMp across a 2×4 GPU process grid. (a) The top-left-aligned matrix is partitioned into $T_A \times T_A$ tiles. (b) Tile-column slabs are redistributed cyclically within each process row, assigning global tile column j to process column $j \bmod P_c$. (c) Tile-row slabs are subsequently redistributed within each process column, assigning global tile row i to process row $i \bmod P_r$. Colours indicate the original GPU ownership of each matrix entry, while grey cells denote padding.

During each cycle, a slab is packed into the send scratch slot, transferred into the receive scratch slot, and unpacked into its destination, while a third saved slot preserves data that would otherwise be overwritten before it is forwarded. Let N_r and N_c denote the local row and column capacities, and let T_r and T_c denote the tile dimensions. Since a tile-column slab contains at most $T_c N_r$ elements and a tile-row slab contains at most $T_r N_c$ elements, the bounded scratch allocation is

$$S_{\text{scratch}} = 3 \max(T_c N_r, T_r N_c)$$

elements, where the factor of three accounts for the send, receive, and saved slots.

Redistribution orchestration

A central design choice in JAXMg is to build the native backend against the matching XLA source through Bazel. This gives the FFI handler access to the underlying NCCL communicator handle (`nccComm_t`) owned by XLA. JAXMg borrows this communicator for both intra-node and inter-node redistribution before passing the same handle to cuSOLVERMp for the distributed solver operation. Movements confined to one rank use local CUDA operations on the XLA-provided stream. The complete forward redistribution, solver operation, and reverse redistribution can therefore share one communication context within a single fused C++/CUDA FFI call.

Research Impact Statement

Scientific applications

We first highlight a selection of examples from across physics in which the dense linear algebra routines provided by JAXMg can be embedded within wider workflows, allowing scientific applications to scale beyond the capabilities of a single GPU while remaining within the JAX ecosystem:

Cholesky solve (potrs)

Across statistical inference, positive-definite linear solves are ubiquitous, particularly in Gaussian processes, a widely used Bayesian nonparametric model, and in the marginalization of nuisance

parameters. For a Gaussian process with observations $\mathbf{y}$, mean vector $\mathbf{m}$, kernel covariance $\mathbf{K}$, and noise covariance Σ , the marginal log likelihood is:

$$\log \mathcal{L} = -\frac{1}{2}(\mathbf{y} - \mathbf{m})^\top (\mathbf{K} + \Sigma)^{-1} (\mathbf{y} - \mathbf{m}) - \frac{1}{2} \log \det(\mathbf{K} + \Sigma) - \frac{N_{\text{data}}}{2} \log(2\pi).$$

The quadratic and determinant terms thus require a solve against the $N_{\text{data}} \times N_{\text{data}}$ matrix $\mathbf{K} + \Sigma$ and its log determinant; analytically integrating linear nuisance parameters with Gaussian priors yields analogous operations for their posterior precision matrix. One application in which both requirements arise is cosmological inference of the 21-cm brightness-temperature field. As next-generation radio interferometers such as the Square Kilometre Array (SKA) come online, JAXMg's distributed Cholesky factorization allows such analyses to accommodate exascale datasets (Liu et al., 2026) and higher-fidelity forward models with larger sets of nuisance parameters (Burba et al., 2023) while remaining within JAX.

Positive-definite solves also arise in stochastic reconfiguration (SR), a natural-gradient method widely used to optimize variational quantum states in variational Monte Carlo (Sorella, 1998). For a state ψ_θ , SR determines the parameter update $\delta\theta$ from

$$(\mathbf{S} + \lambda \mathbf{I}) \delta\theta = \mathbf{F},$$

where $\mathbf{S}$ is the quantum geometric tensor, $\mathbf{F}$ is the variational force associated with the energy gradient, and $\lambda > 0$ is a diagonal regularization. The Monte Carlo estimate of $\mathbf{S}$ is Hermitian positive semidefinite; the regularization makes it positive definite and therefore suitable for Cholesky factorization. Because this solve is repeated at every optimization step, explicitly forming $\mathbf{S}$ for N_{par} parameters requires an $N_{\text{par}} \times N_{\text{par}}$ dense matrix and can quickly become limited by single-GPU memory. Alternatively, in the MinSR formulation (Chen & Heyl, 2024; Rende et al., 2024), the neural tangent kernel is of size $N_{\text{samples}} \times N_{\text{samples}}$, which faces a similar problem as the number of Monte Carlo samples increases. JAXMg's distributed potrs allows the parameter update to remain within one JIT-compiled JAX workflow, enabling direct stochastic-reconfiguration solves for larger neural quantum states and other parameterized variational ansätze. JAXMg is integrated in the latest version of NetKet (Vicentini et al., 2022).

General linear solve (lu_solve)

Across computational electromagnetics, general linear solves are central to antenna simulations that model electromagnetic fields and their interaction with complex structures through numerical solutions of Maxwell's equations. A common approach is the method of moments (Harrington, 1993), in which the governing integral equations are discretized to produce the dense complex system $\mathbf{Z}\mathbf{I} = \mathbf{V}$, where $\mathbf{Z}$ is the impedance matrix, $\mathbf{I}$ contains the unknown current coefficients, and $\mathbf{V}$ represents one or more excitations. Recovering the current coefficients therefore requires solving this general nonsingular system. One application in which this becomes computationally demanding is the full-wave simulation of mutual coupling across the 320-element core of the Hydrogen Epoch of Reionization Array for 21-cm cosmology (Gueuning et al., 2026). As precision requirements drive larger arrays and finer geometric meshes, the number of basis functions and hence the dimensions of $\mathbf{Z}$ grow. JAXMg's distributed LU factorization allows the construction of $\mathbf{Z}$, the solution of $\mathbf{Z}\mathbf{I} = \mathbf{V}$, and downstream analysis to remain within a single compiled JAX workflow, supporting efficient end-to-end simulation at increasing model fidelity.

Singular-value decomposition (gesvd)

Principal component analysis (PCA) uses the singular-value decomposition to identify dominant directions of variation in high-dimensional data. For a centered data matrix $\mathbf{X} \in \mathbb{R}^{N_{\text{samples}} \times N_{\text{features}}}$,

$$X = U\Sigma V^T,$$

the columns of V are the principal directions, while $U\Sigma$ contains the corresponding low-dimensional representations of the samples. The variance associated with the i th component is proportional to σ_i^2 , allowing the data to be compressed by retaining only the leading singular values and vectors. JAXMg's distributed gesvd routine enables PCA for dense datasets whose sample or feature dimensions make the full data matrix and its decomposition too large for a single GPU.

Singular-value decompositions are a central primitive in tensor-network algorithms for quantum many-body systems (Bañuls, 2023; Schollwöck, 2011). After applying a local gate, contracting neighbouring tensors, or enlarging an auxiliary bond, the resulting tensor is reshaped into a matrix and decomposed as $X = U\Sigma V^T$. Retaining the largest singular values yields the optimal low-rank approximation in the Frobenius norm, controls the tensor-network bond dimension, and provides an estimate of the truncation error through the discarded singular-value weight. The SVD is therefore the backbone of modern tensor-network algorithms such as the density matrix renormalization group (DMRG), time-evolving block decimation (TEBD), and approximate contraction schemes for high-dimensional networks. As the physical dimension and bond dimensions grow, the intermediate matrices entering these decompositions can become considerably larger than the tensors retained after truncation and may exceed the memory of one GPU. JAXMg's distributed gesvd allows tensor contractions, reshaping, singular-vector construction, and truncation to remain within a JIT-compiled JAX workflow, supporting tensor-network calculations at larger bond dimensions without exporting arrays to an external distributed solver.

Symmetric eigendecomposition (syevd)

The time-dependent variational principle (TDVP) projects Schrödinger evolution onto the tangent space of a parameterized quantum state, producing equations of motion of the form (Carleo & Troyer, 2017; Schmitt & Heyl, 2020)

$$\dot{\theta} = -iS^+F,$$

where S and F are the same objects as in the Cholesky-solve section. In practical time-dependent variational Monte Carlo calculations, redundant parameters, gauge directions, and Monte Carlo noise can make S singular or severely ill-conditioned. An eigendecomposition $S = V\Lambda V^\dagger$ exposes these directions and permits stable spectral regularization, for example by discarding eigenvalues below a threshold or replacing Λ^{-1} with a smooth pseudoinverse. Since the decomposition may be required at every stage of an ODE integrator, its memory and computational cost become substantial as the number of variational parameters grows. JAXMg's distributed syevd enables full-spectrum regularization and diagnostics to be carried out on matrices exceeding single-GPU memory while keeping state evaluation, Monte Carlo estimation, and time integration inside a compiled JAX program. JAXMg has been used for the t-VMC simulations of Refs. (Wan et al., 2026; Wiersema, 2026).

Performance and scaling

Performance in distributed matrix routines depends not only on the available GPU resources, but also on how the calculation is divided between them. To quantify this dependence and provide practical guidance for users, we investigate the effects of the process-grid layout and matrix tile dimension, T_A , on wall-clock runtime in Figures 3 and 4, respectively.

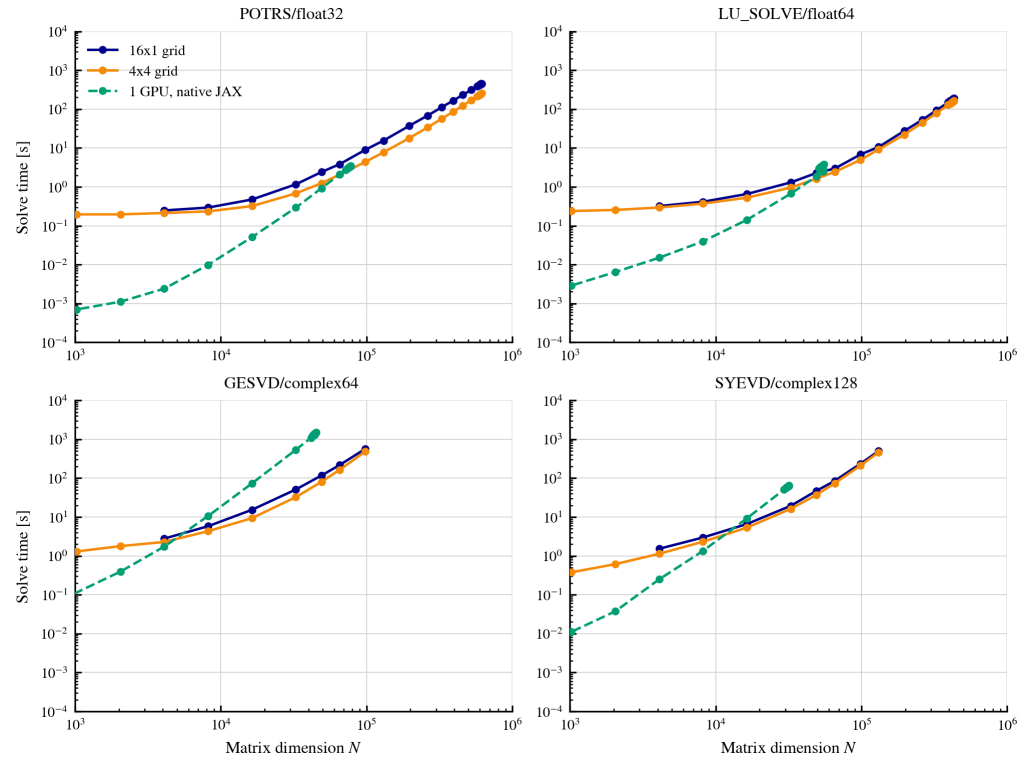


Figure 3: Wall-clock runtime against matrix dimension N for JAXMg on 16 NVIDIA H100 GPUs at fixed tile dimension $T_A = 256$. The 16×1 process grid is shown in dark blue, the 4×4 process grid in orange, and native single-GPU JAX in dashed green. The panels show (a) `jaxmg.potrs` (float32), (b) `jaxmg.lu_solve` (float64), (c) `jaxmg.gesvd` (complex64), and (d) `jaxmg.syevd` (complex128). The 16×1 curves begin at larger N because avoiding padding requires N to be divisible by $16T_A$, compared with $4T_A$ for the 4×4 grid.

We consider four representative combinations of routine and data type: `potrs` with float32, `lu_solve` with float64, `gesvd` with complex64, and `syevd` with complex128. For `potrs`, `lu_solve`, and `syevd`, we use the diagonal matrix $A = \text{diag}(1, \dots, N)$; for the two linear solves, we additionally set $b = (1, \dots, 1)^T$. For `gesvd`, we use a random Gaussian matrix with zero mean and unit variance. Both benchmarks use 16 NVIDIA H100 SXM5 GPUs (94 GB each) across four nodes, with four NVLink-connected GPUs and four 400 Gb/s InfiniBand links per node. The benchmark implementation is available at (Tutt & Wiersema, 2026).

Figure 3 compares a one-dimensional 16×1 process grid with a two-dimensional 4×4 grid using a fixed tile dimension $T_A = 256$. Under the block-cyclic redistribution outlined in Section 3.1.3, moving from the 16×1 to the 4×4 grid introduces a second inter-rank redistribution phase, but provides a more balanced two-dimensional distribution during cuSOLVERMp execution. The process-grid layout therefore determines the balance between redistribution overhead and distributed solver efficiency. The results show that, across every routine, the solver-side benefit outweighs the additional communication, with the 4×4 grid achieving geometric-mean speed-ups over the matrix dimensions completed by both layouts of $1.77\times$ for `potrs`, $1.23\times$ for `lu_solve`, $1.38\times$ for `gesvd`, and $1.21\times$ for `syevd` relative to the 16×1 grid. The smaller maximum matrix dimensions reached by `gesvd` and `syevd` reflect the significantly larger cuSOLVERMp workspaces required by these decompositions. Finally, Figure 3 demonstrates that, for each representative routine-dtype combination, both distributed layouts outperform the corresponding native single-GPU JAX implementation at sufficiently large N , showing that the computational benefit of distributed execution outweighs its redistribution overhead before single-GPU memory becomes limiting.

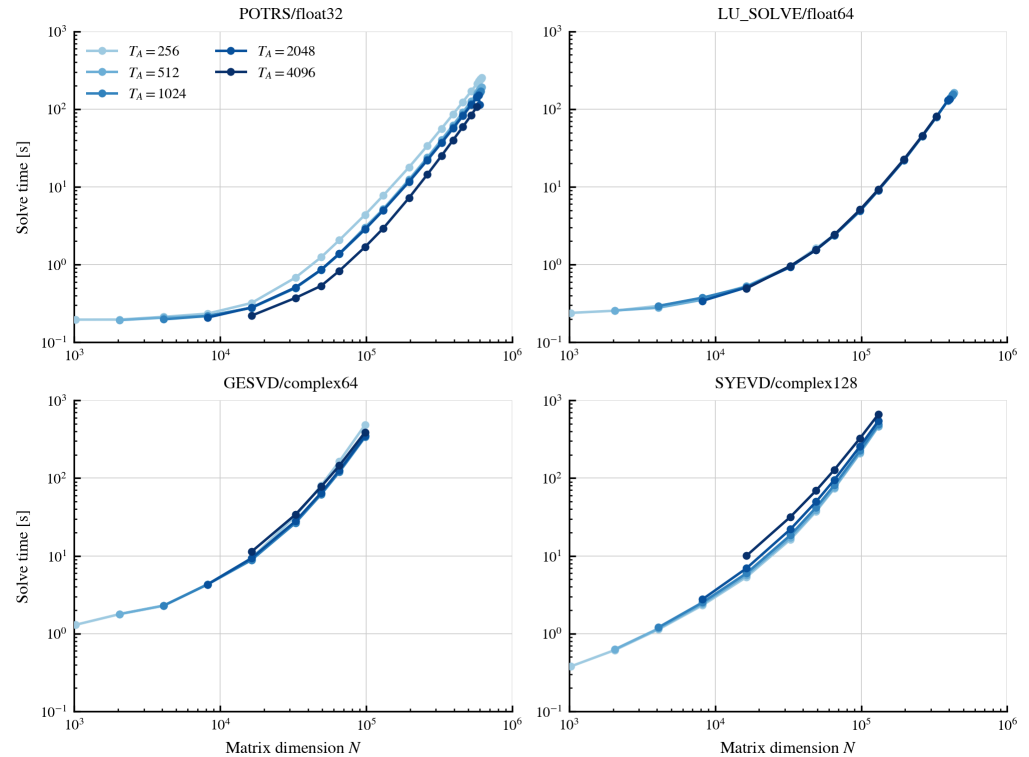


Figure 4: Effect of the tile size on JAXMg wall-clock runtime for the 4×4 process grid on the same 16 NVIDIA H100 GPUs. Each panel draws one curve per $T_A \in \{256, 512, 1024, 2048, 4096\}$, shaded light to dark with increasing tile size. There are four panels: (a) `jaxmg.potrs` (float32), (b) `jaxmg.lu_solve` (float64), (c) `jaxmg.gesvd` (complex64) and (d) `jaxmg.syevd` (complex128). Curves for larger T_A begin at larger N because a dimension must be a whole number of tiles along both grid axes.

Having established the performance advantage of the 4×4 process grid, we next hold this layout fixed and vary the tile dimension from $T_A = 2^8$ to 2^{12} , as shown in Figure 4. The response is strongly routine-dependent: increasing T_A reduces the runtime of `potrs`, leaves `lu_solve` broadly unchanged, and increases the runtime of `syevd`, while `gesvd` is non-monotonic, running fastest at an intermediate $T_A = 1024$ and more slowly for both smaller and larger tiles.

We finally demonstrate the large-scale performance of JAXMg using 64 NVIDIA H200 SXM5 GPUs (143 GB each) across eight nodes, with eight NVLink-connected GPUs and eight 400 Gb/s InfiniBand links per node. By reducing the redistribution scratch required for a fixed tile dimension relative to the 64×1 layout, the balanced 8×8 process grid enabled a Cholesky solve with $N = 1,499,136$ and $T_A = 1024$. The distributed matrix occupied 8.2 TiB in aggregate, while each local shard and its redistribution scratch used 94.7% of the available device memory. The warm solve completed in 654 seconds. At $N = 1,310,720$, the largest dimension completed by both layouts, the 8×8 grid was also $6.8\times$ faster, requiring 452 seconds compared with 3065 seconds for 64×1 .

These results highlight JAXMg's primary impact: enabling dense matrix solves and decompositions that are bottlenecked by the memory capacity of a single GPU, while remaining within JAX's composable and JIT-compiled programming model. On modern multi-GPU systems, distributed solvers make it possible to tackle matrix sizes that would otherwise be infeasible, and to increase throughput by using aggregate device memory and compute.

AI usage disclosure

Claude Code was used during software development for code exploration and debugging. Large language models were used to assist with language polishing.

Acknowledgements

We want to thank Dennis Bollweg, Alex Chavin, Geraud Krawezik, Dylan Simon and Nils Wentzell for their help with developing the code. We also want to acknowledge the help of Ao Chen and Riccardo Rende with testing the code in applied settings. RW is grateful to Simon Tartakovsky for his suggestions on the 1D cyclic algorithm. Finally, we want to thank Filippo Vincentini for his suggestions on code distribution. RW acknowledges support from the Flatiron Institute. The Flatiron Institute is a division of the Simons Foundation. JT is supported by the Harding Distinguished Postgraduate Scholars Programme (HDPSP) and the Science and Technology Facilities Council (STFC) DTP Studentship.

The authors acknowledge the use of resources provided by the Isambard-AI National AI Research Resource (AIRR). Isambard-AI (McIntosh-Smith et al., 2024) is operated by the University of Bristol and is funded by the UK Government's Department for Science, Innovation and Technology (DSIT) via UK Research and Innovation; and the Science and Technology Facilities Council [ST/AIRR/I-A-I/1023].

References

- Abdelfattah, A., Beams, N., Carson, R., Ghysels, P., Kolev, T., Stitt, T., Vargas, A., Tomov, S., & Dongarra, J. (2024). MAGMA: Enabling exascale performance with accelerated BLAS and LAPACK for diverse GPU architectures. *The International Journal of High Performance Computing Applications*, 38(5), 468–490. <https://doi.org/10.1177/10943420241261960>
- Bañuls, M. C. (2023). Tensor network algorithms: A route map. *Annual Review of Condensed Matter Physics*, 14(1), 173–191.
- Blackford, L. S., Choi, J., Cleary, A., D'Azevedo, E., Demmel, J., Dhillon, I., Dongarra, J., Hammarling, S., Henry, G., Petitet, A., & others. (1997). *ScaLAPACK users' guide*. SIAM.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). <http://github.com/google/jax>
- Burba, J., Sims, P. H., & Pober, J. C. (2023). All-sky modelling requirements for Bayesian 21 cm power spectrum estimation with bayeseor. *Monthly Notices of the Royal Astronomical Society*, 520(3), 4443–4455. <https://doi.org/10.1093/mnras/stad401>
- Cabezas, A., Corenflos, A., Lao, J., & Louf, R. (2024). *BlackJAX: Composable Bayesian inference in JAX*. <https://arxiv.org/abs/2402.10797>
- Carleo, G., & Troyer, M. (2017). Solving the quantum many-body problem with artificial neural networks. *Science*, 355(6325), 602–606. <https://doi.org/10.1126/science.aag2302>
- Catanzaro, B., Keller, A., & Garland, M. (2014). A decomposition for in-place matrix transposition. *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 193–206. <https://doi.org/10.1145/2555243.2555253>
- Chen, A., & Heyl, M. (2024). Empowering deep neural quantum states through efficient optimization. *Nature Physics*, 20(9), 1476–1481.
- Corporation, N. (n.d.). *cuSOLVERmp API*. <https://docs.nvidia.com/cuda/cusolvermp/index>.

- 328 [html](#)
- 329 Dongarra, J. J., Vandegeijn, R. A., & Walker, D. W. (1994). Scalability Issues Affecting the
330 Design of a Dense Linear Algebra Library. *Journal of Parallel and Distributed Computing*,
331 22(3), 523–537. <https://doi.org/10.1006/jpdc.1994.1108>
- 332 Freeman, C. D., Frey, E., Raichuk, A., Girgin, S., Mordatch, I., & Bachem, O. (2021). *Brax -*
333 *A Differentiable Physics Engine for Large Scale Rigid Body Simulation* (Version 0.14.0).
334 <http://github.com/google/brax>
- 335 Gates, M., Kurzak, J., Charara, A., YarKhan, A., & Dongarra, J. (2019). SLATE: Design of a
336 modern distributed and accelerated linear algebra library. *Proceedings of the International*
337 *Conference for High Performance Computing, Networking, Storage and Analysis*, 1–18.
- 338 Gueuning, Q., Lera Acedo, E. de, Brown, A. K., & Fagnoni, N. (2026). *A Fast Direct Solver*
339 *for Mutual Coupling Analysis of Large Arrays of Reflector Antennas*. [https://arxiv.org/](https://arxiv.org/abs/2604.10239)
340 [abs/2604.10239](https://arxiv.org/abs/2604.10239)
- 341 Harrington, R. F. (1993). *Field Computation by Moment Methods*. Wiley–IEEE Press.
- 342 Heek, J., Levskaya, A., Oliver, A., Ritter, M., Rondepierre, B., Steiner, A., & Zee, M.
343 van. (2024). *Flax: A neural network library and ecosystem for JAX* (Version 0.12.2).
344 <http://github.com/google/flax>
- 345 Kidger, P. (2021). *On Neural Differential Equations* [PhD thesis]. University of Oxford.
- 346 Li, A., Song, S. L., Chen, J., Li, J., Liu, X., Tallent, N. R., & Barker, K. J. (2020). Evaluating
347 modern GPU interconnect: PCIe, NVLink, NV-SLI, NVSwitch and GPUDirect. *IEEE*
348 *Transactions on Parallel and Distributed Systems*, 31(1), 94–110. [https://doi.org/10.1109/](https://doi.org/10.1109/TPDS.2019.2928289)
349 [TPDS.2019.2928289](https://doi.org/10.1109/TPDS.2019.2928289)
- 350 Liu, Y., Lera Acedo, E. de, & Sims, P. H. (2026). Bayesian model comparison and validation
351 with Gaussian Process Regression for interferometric 21-cm signal recovery. *Monthly*
352 *Notices of the Royal Astronomical Society*. <https://doi.org/10.1093/mnras/stag878>
- 353 McIntosh-Smith, S., Alam, S. R., & Woods, C. J. (2024). Isambard-AI: A leadership-class
354 supercomputer optimised specifically for artificial intelligence. *CUG 2024 Proceedings*,
355 44–54.
- 356 Rader, J., Lyons, T., & Kidger, P. (2023). Lineax: Unified linear solves and linear least-squares
357 in JAX and equinox. *AI for Science Workshop at Neural Information Processing Systems*.
358 <https://arxiv.org/abs/2311.17283>
- 359 Rende, R., Viteritti, L. L., Bardone, L., Becca, F., & Goldt, S. (2024). A simple linear algebra
360 identity to optimize large-scale neural network quantum states. *Communications Physics*,
361 7(1), 260.
- 362 Schmitt, M., & Heyl, M. (2020). Quantum many-body dynamics in two dimensions with
363 artificial neural networks. *Physical Review Letters*, 125(10), 100503. [https://doi.org/10.](https://doi.org/10.1103/PhysRevLett.125.100503)
364 [1103/PhysRevLett.125.100503](https://doi.org/10.1103/PhysRevLett.125.100503)
- 365 Schollwöck, U. (2011). The density-matrix renormalization group in the age of matrix product
366 states. *Annals of Physics*, 326(1), 96–192.
- 367 Sorella, S. (1998). Green function monte carlo with stochastic reconfiguration. *Physical Review*
368 *Letters*, 80(20), 4558.
- 369 Tutt, J., & Wiersema, R. (2026). *JAXMg: Benchmarks*. [https://github.com/flatironinstitute/](https://github.com/flatironinstitute/jaxmg_benchmark)
370 [jaxmg_benchmark](https://github.com/flatironinstitute/jaxmg_benchmark)
- 371 Vicentini, F., Hofmann, D., Szabó, A., Wu, D., Roth, C., Giuliani, C., Pescia, G., Nys,
372 J., Vargas-Calderón, V., Astrakhantsev, N., & Carleo, G. (2022). NetKet 3: Machine
373 Learning Toolbox for Many-Body Quantum Systems. *SciPost Phys. Codebases*, 7. [https://](https://doi.org/10.21468/scipostphyscodebases.7.01.001)

374 [//doi.org/10.21468/SciPostPhysCodeb.7](https://doi.org/10.21468/SciPostPhysCodeb.7)

375 Wan, Z.-Q., Wiersema, R., & Zhang, S. (2026). Removing nodal and support-mismatch
376 pathologies in variational monte carlo via blurred sampling. *Phys. Rev. X*, 16, 031059.
377 <https://doi.org/10.1103/jrn5-gv19>

378 Wiersema, R. (2026). *Numerical simulation of D-Wave's quantum advantage experiment with*
379 *time-dependent variational Monte Carlo*. <https://arxiv.org/abs/2609.01719>

DRAFT